

Komplexní výuková příručka

Principy a implementace automatizované detekce a klasifikace mincí pomocí počítačového vidění a hlubokého učení

Úvod

Vítejte v této rozsáhlé výukové příručce, která je koncipována jako vyčerpávající průvodce světem automatizované analýzy obrazu se zaměřením na specifickou a pedagogicky vděčnou úlohu: **detekci a rozpoznávání mincí**. Tato úloha v sobě unikátním způsobem snoubí dvě fundamentální disciplíny moderní umělé inteligence – **klasické počítačové vidění (Computer Vision)**, zaměřené na geometrickou analýzu a zpracování signálu, a **hluboké učení (Deep Learning)**, které přináší schopnost sémantické klasifikace a generalizace.

Cílem tohoto textu není pouze poskytnout funkční kód, ale především vybudovat hluboké teoretické porozumění principům, které umožňují strojům **"vidět"** a **"chápat"** vizuální data.

Numismatika a automatizované třídění mincí představují pro počítačové vidění zajímavou výzvu. Mince jsou objekty s pevně danou geometrií (kruhový tvar), což umožňuje využití robustních matematických transformací, avšak jejich povrch je vysoce variabilní. Různé stupně opotřebení, oxidace (patina), odlesky kovu, variabilní osvětlení a jemné detaily ražby činí z klasifikace netriviální problém, který nelze řešit pouhým porovnáváním pixelů.

Historicky vyžadovala identifikace mincí expertní znalosti a manuální kontrolu, což bylo časově náročné a subjektivní. S příchodem konvolučních neuronových sítí (CNN) a pokročilých algoritmů segmentace se však otevírají možnosti pro plnou automatizaci.

Tato příručka je strukturována do logických bloků, které kopírují reálný tok dat v aplikaci počítačového vidění: od pořízení obrazu, přes jeho předzpracování a segmentaci, až po extrakci příznaků a finální klasifikaci pomocí neuronových sítí. U každého tématu budeme důsledně rozlišovat mezi teoretickým principem (matematickým a algoritmickým pozadím) a praktickou implementací (kódem v Pythonu s využitím knihoven OpenCV a PyTorch).

Důraz bude kladen na pochopení kauzálních souvislostí – proč volíme určité parametry, proč nahrazujeme vlastní modely transfer learningem a jak interpretovat chování neuronové sítě během tréninku.

Kapitola 1: Digitální reprezentace obrazu a předzpracování dat

Než se pustíme do detekce mincí, musíme pochopit, s jakými daty pracujeme. Digitální obraz, ačkoliv se lidskému oku jeví jako spojitá scéna, je pro počítač diskrétní maticí číselných hodnot. Pochopení této reprezentace je klíčové pro všechny následné operace.

1.1 Teoretický princip: Obraz jako matice a barevné prostory

V kontextu počítačového vidění je rastrový obraz (bitmapa) reprezentován jako mřížka pixelů. Každý pixel nese informaci o intenzitě světla v daném bodě. V nejběžnějším barevném modelu **RGB (Red, Green, Blue)** je

1.1 Teoretický princip: Obraz jako matice a barevné prostory

V kontextu počítačového vidění je rastrový obraz (bitmapa) reprezentován jako mřížka pixelů. Každý pixel nese informaci o intenzitě světla v daném bodě. V nejběžnějším barevném modelu **RGB (Red, Green, Blue)** je každý pixel definován trojicí hodnot, obvykle v rozsahu 0 až 255 (pro 8bitovou hloubku), kde **0 představuje absenci barvy (černá)** a **255 plnou intenzitu**. Barevný obraz o rozměrech $H \times W$ (výška $\times$ šířka) je tedy matematicky reprezentován jako **tenzor** o rozměrech $H \times W \times 3$.

Pro úlohu detekce tvarů, jako jsou mince, je však barva často redundantní a může dokonce vnášet do analýzy šum (například různé odlesky na zlaté a stříbrné minci mohou mást detektory hran). Proto je prvním krokem v našem řetězci převod do stupňů šedi (**grayscale**). V tomto jedkanálovém režimu ($H \times W \times 1$) reprezentuje hodnota pixelu jas (luminanci). Převod se obvykle neprovádí prostým průměrem RGB kanálů, ale **váženým součtem**, který zohledňuje citlivost lidského oka na různé vlnové délky světla (oko je nejcitlivější na zelenou):

$$Y = 0.299R + 0.587G + 0.114B$$

Tento proces zjednodušuje data (redukce objemu dat na třetinu) a zdůrazňuje strukturální informace (hrany, texturu) na úkor chromatických informací.

1.2 Redukce šumu a teorie filtrace

Digitální fotografie mincí nejsou nikdy dokonalé. Obsahují šum, který může pocházet z tepelného šumu senzoru kamery nebo textury podkladu. **Detektory hran, které jsou založeny na derivacích (změnách jasu), jsou na šum extrémně citlivé.**

Proto je nezbytné obraz **vyhladit (rozostřit)**.

Mediánový filtr vs. Gaussovský filtr

V naší finální implementaci (`utils.py`) používáme **Gaussovské rozostření (Gaussian Blur)** o velikosti jádra 5×5 . Ačkoliv mediánový filtr lépe zachovává hrany, Gaussovský filtr je výpočetně efektivnější a pro Cannyho detektor hran (který má vyhlazování integrované) je přirozenějším vstupem, jelikož Cannyho algoritmus je založen na derivacích Gaussovy funkce. Jádro 5×5 s $\sigma \approx 1.5$ poskytuje ideální kompromis mezi potlačením šumu koberce/stolu a zachováním obrysu mince.

Kapitola 2: Detekce hran a gradientní analýza

Po vyhlazení obrazu je naším cílem nalézt hranice objektů. Hrana v digitálním obraze je definována jako oblast s výraznou změnou jasu (diskontinuitou intenzitní funkce).

2.1 Algoritmus Canny Edge Detector

Pro detekci hran mincí je průmyslovým standardem Cannyho detektor. Není to pouhý konvoluční filtr, ale vícestupňový algoritmus:

- Gaussovské vyhlazení:** Redukce šumu.
- Výpočet gradientů:** Pomocí Sobelova operátoru určí sílu a směr hrany.
- Potlačení nemaximálních hodnot (Non-Maximum Suppression):** Ztenčení hran na šířku 1 pixelu.
- Hysterezní prahování (Hysteresis Thresholding):** Použití dvou prahů (T_{min} , T_{max}). Silné hrany ($> T_{max}$) jsou zachovány, slabé hrany ($T_{min} < h < T_{max}$) jsou zachovány pouze pokud navazují na silné.

Tento mechanismus je pro mince zásadní. Odlesk na hraně mince může být v některých místech silný a jinde slabý. Hystereze umožní detekovat celý obvod mince, pokud je alespoň část hrany výrazná.

Tento mechanismus je pro mince zásadní. Odlesk na hraně mince může být v některých místech silný a jinde slabý. Hystereze umožní detekovat celý obvod mince, pokud je alespoň část hrany výrazná.

Kapitola 3: Geometrická detekce kružnic (Houghova transformace)

Máme-li obraz hran, dalším úkolem je nalézt v těchto hranách geometrické primitivum – kružnici. K tomu slouží **Houghova transformace**.

3.1 Hough Gradient Method: Optimalizace pro praxi

OpenCV implementuje metodu **HOUGH_GRADIENT**. Tato metoda snižuje dimenzionalitu problému (z 3D prostoru parametrů a, b, r na 2D) využitím směru gradientu.

Algoritmus pracuje ve dvou fázích:

1. **Detekce středů:** Využívá faktu, že gradient na obvodu kruhu směřuje do středu. Akumulátor sčítá průsečíky normál hran.
2. **Odhad poloměru:** Kolem nalezených center hledá nejčastěji se vyskytující vzdálenost k hranovým bodům.

3.2 Praktická implementace a "Bimetalický problém"

V naší aplikaci (`utils.py`) používáme funkci `detect_regions` . Zde jsme narazili na kritický problém s bimetalickými mincemi (např. 50 Kč). Detektor často našel dva kruhy: jeden menší (vnitřní zlatý střed) a jeden větší (vnější měděný okraj).

Řešení v kódu:

```
# Seřazení kružnic podle poloměru (SESTUPNĚ)
# Preferujeme větší vnější kružnice, abychom zachytili celou minci.
sorted_indices = np.argsort(circles[0, :, 2])[:-1]
sorted_circles = circles[0, sorted_indices, :]
```

Díky řazení od největšího poloměru a následné filtraci duplicátů (pokud má nový kruh střed blízko již vybraného většího kruhu, ignorujeme ho) zajistíme, že aplikace správně vyřizne celou 50 Kč minci, což je klíčové pro klasifikaci.

Kapitola 4: Hluboké učení a Transfer Learning (ResNet18)

Zatímco Houghova transformace mince **najde**, neuronová síť je musí **pojmenovat**. Zde jsme přešli od vlastních jednoduchých architektur (které dosahovaly nízké přesnosti ~45 %) k profesionální technice zvané **Transfer Learning**.

4.1 Proč Transfer Learning?

Trénovat konvoluční síť (CNN) od nuly na malém datasetu (stovky mincí) je obtížné. Síť se musí naučit vnímat

4.1 Proč Transfer Learning?

Trénovat konvoluční síť (CNN) od nuly na malém datasetu (stovky mincí) je obtížné. Síť se musí naučit vnímat hrany, rohy, textury a tvary "od píky".

Použili jsme model **ResNet18** (Residual Network s 18 vrstvami), který byl předtrénován na datasetu **ImageNet** (1.2 milionu obrázků, 1000 kategorií). Tento model již "umí vidět". Jeho filtry jsou vyladěné na detekci vizuálních primitiv. My pouze "přeučíme" jeho poslední vrstvu, aby místo psů a aut poznávala české mince.

Díky tomu jsme dosáhli skoku v přesnosti z **45 %** na **>95 %**.

4.2 Implementace v PyTorch (`model.py` , `train.py`)

Pro správné fungování Transfer Learningu je nutné dodržet přesný postup přípravy dat, na který byl původní model zvyklý.

Normalizace vstupu

ResNet18 vyžaduje, aby vstupní obrázky byly normalizovány pomocí specifických hodnot průměru a směrodatné odchylky datasetu ImageNet:

```
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

Pokud tento krok vynecháme (jak se stalo v dřívějších verzích projektu), síť dostává data v jiném rozsahu, než očekává, a její výkon se hroutí.

Augmentace dat pro robustnost

Mince může být na stole libovolně natočená. Aby se model naučil, že "pětka hlavou dolů" je stále pětka, používáme při tréninku (`train.py`) silnou augmentaci:

- **RandomRotation(180)**: Klíčové pro rotační invarianci.
- **ColorJitter**: Mění jas a kontrast (simulace světla), ale jen velmi opatrně mění odstín (Hue), aby si model nepletl "zlatou" 20 Kč a "měděnou" 10 Kč.
- **Padding**: Při extrakci mince (`utils.py`) přidáváme 30 % okraj, aby síť viděla hranu mince v kontextu.

Kapitola 5: Implementační příručka (Code Walkthrough)

V této části spojíme teorii do funkčního celku. Použijeme hybridní přístup: OpenCV pro hledání a PyTorch (ResNet18) pro klasifikaci.

7.1 Krok 1: Lokalizace a extrakce (`utils.py`)

Nejprve nalezneme kruhy, seřadíme je sestupně a vyřízneme s paddingem.

Nejprve nalezneme kruhy, seřadíme je sestupně a vyřízneme s paddingem.

```
def extract_coin_image(image_cv, center_x, center_y, radius, target_size=(128, 128)):
    # Padding 30 % zajistí, že vidíme i okraj mince
    padding = int(radius * 0.3)
    x1 = max(0, center_x - radius - padding)
    # ... oříznutí ...
    crop_resized = cv2.resize(crop, target_size)
    return cv2.cvtColor(crop_resized, cv2.COLOR_BGR2RGB)
```

7.2 Krok 2: Klasifikace Modelem (`app.py`)

Vstupní obrázek (výřez) musíme převést na tenzor a normalizovat.

```
# Načtení modelu
net = model.load_model() # Načte náš ResNet18

# Pipeline transformací (shodná s tréninkem!)
transform_pipeline = transforms.Compose([
    transforms.ToTensor(), # Převed na hodnoty 0.0 - 1.0
    transforms.Normalize(mean=[0.485,...], std=[0.229,...]) # ImageNet statistiky
])

# Inference
with torch.no_grad():
    outputs = net(transform_pipeline(crop).unsqueeze(0))
    probs = F.softmax(outputs, dim=1) # Získání procentuální jistoty
```

Při nízké jistotě (< 30-40 %) výsledek zahazujeme, abychom neukazovali nesmysly.

Závěr

Tento projekt demonstruje sílu moderního přístupu k počítačovému vidění. Kombinace klasických algoritmů (Houghova transformace) pro precizní lokalizaci a hlubokých neuronových sítí (ResNet + Transfer Learning) pro robustní sémantickou klasifikaci řeší úlohu rozpoznávání mincí s vysokou spolehlivostí.

Klíčovým poznatkem je, že **kvalita dat a jejich předzpracování** (zachování poměru stran, správný ořez, normalizace) je často důležitější než samotná volba architektury neuronové sítě.